

ECE 1387 - lecture 7 Oct 24, 2025

- AI marked
- Ex 1 to be posted
- Quiz 1 end of next lecture #8
 - examples posted
- No class next week!
- Go Blue Jays!

cct1

distrib

2/35 AP, 2V

top bottom

2/36 AH

? SL

cct2

3/120 AH

3/164 CM

cct3

4/708 AD

4/730 LQ

cct4

5/2560 LQ

6/2286 2V

The Pieces of SA:

Starting temp — "hot" enough so most moves would be accepted, e.g. 90%

$$\rightarrow \text{"acceptance rate"} = \frac{\# \text{ accepted}}{\# \text{ total moves}}$$

— make say N moves
get ΔC for each, choose *typical way*
To, so 90% of N would be accepted.

Stopping Criteria: way of deciding when to stop SA.

— could look at how cost is changing over last temp, or last several temps

⇒ terminate if cost not improving (much).

inner-loop criteria: way

to decide when to lower T

— could use statistics of Δc values to make a decision

— could just do a fixed #

of moves per temp

$$\hookrightarrow \# \text{ moves}/T = N^{4/3}$$

in VPR
tool -

N of objs
to place

Generate: key is random

- given current state, choose a new one

(e.g. in placement, swap cells)

- want it to be fast to calculate Δc

- want to design it so any state can be reached

Accept: if $\Delta C < 0$, accept

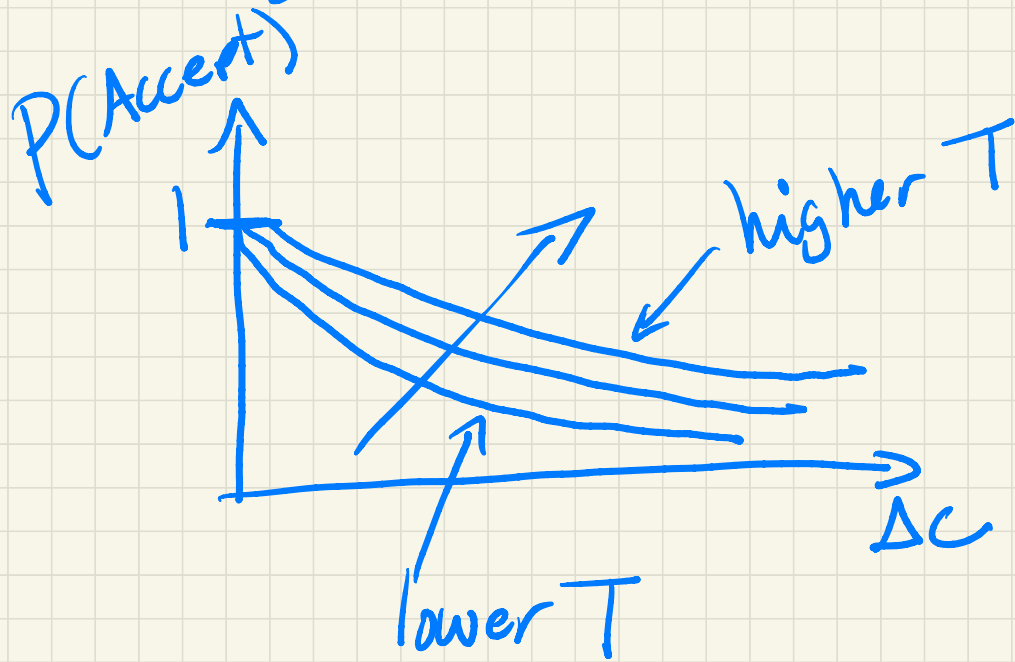
$\hookrightarrow$ these moves ^{always} reduce cost

if $\Delta C \geq 0$, maybe accept

$P(\text{Accept}) \uparrow$ when T is high

$P(\text{Accept}) \downarrow$ when ΔC is very large

Most typical $P(\text{Accept}) = e^{-\frac{\Delta C}{T}}$



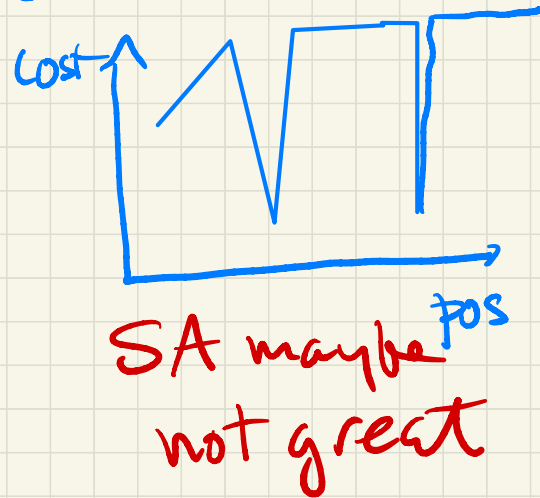
Update T: how much to lower temperature?

$$T_{\text{new}} = \alpha \cdot T_{\text{old}} \quad \left. \begin{array}{l} \text{Common} \\ \text{way} \end{array} \right\} \alpha \text{ some } \# < 1, \text{ eg } \underline{\underline{0.9}}$$

Also, $\exists$ "adaptive" cooling
schedule: adjust T based on
 Δc statistics, acceptance rate,
...

Above is general SA framework
How well it works is a
function of:

i. The nature of the problem



ii) The implementation

i) cost function design
↳ SA "best feature" →
can almost anything

ii) move generation

iii) cooling schedule
↳ how to adjust T

Example SA-based Placement
Implementation : TiberWolf

UC Berkeley 1980's

- standard cell placement

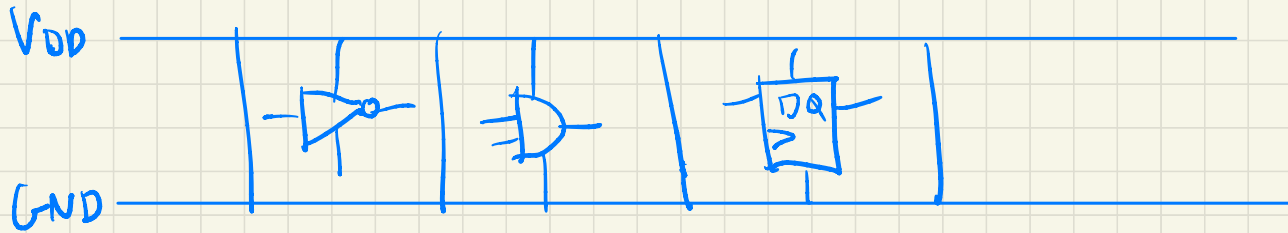
1) Cost Function : HPWL (like in A2)

2) Move generation:
2 types of move

1. Pick a random cell, move a random distance (DISPLACE)
2. pick 2 random cells, swap them (interchange positions) (EXCHANGE)

Standard-cell Design Methodology

- All cells equal height



- Different cells have different width

Issue #1: moving a cell or
interchanging causes overlap

What to do?

1. Shuffle or shift cells to
remove overlap

- can mess up inter-row
connections

2. TW solⁿ → allow overlaps
penalize overlaps
in cost function
$$= K_1 (\text{Amount of overlap})^2$$

Issue #2 : @ low T , unlikely
that a long-range random
move would be accepted

→ meaning, SA not efficient
as most moves are rejected

Solⁿ: TLW implements a range
window . Moves can only
happen in window

Initially, window size is whole
chip

As $T \downarrow$, lower/reduce window size

$\hookrightarrow$ cells can only move in
a local region

Issue #3

- long rows are bad
 - waste Si area
- penalize excess row width in cost function
$$= K_2 \times |\text{width}(\text{row}) - \text{AvgWidth}|$$

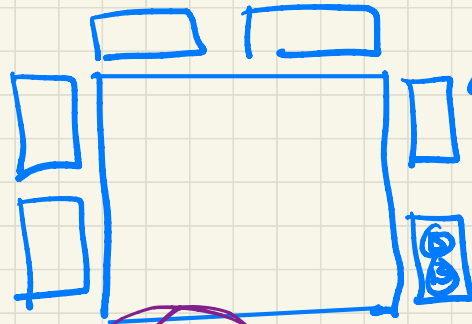
Cost function

$$= \text{HPWL} + k_1 \cdot \text{Overlap Penalty} + k_2 \cdot \text{Row Penalty}$$

k_1, k_2 : weights chosen experimentally

Example SA-based I/O placement

Xilinx
Virtex
FPGA
8 I/O banks



signaling
Standards
Configurable
@ bank
level

(A) I/O with signaling
Standard (A)

I/O bank

Winner = (A)

For each bank z let

$\text{winner}(z)$: most common std in z

$\text{Cost} = \sum_{z \in \text{banks}} \# \text{ of I/Os in } z \text{ incompatible}$
with $\text{winner}(z)$

+ HPWL + perf + ...

Partitioning

Informal definition: break a

circuit into pieces, so

the pieces are as disconnected
as possible

(# of "cut" signals between
pieces) is minimized

Many applications:

- Huge circuit, partition $\rightarrow$
place/route pieces (partitions)
in parallel
- Input to floorplanning
- Multi-FPGA prototyping system
(Synopsys)
 $\rightarrow$ for ASIC verification

- Distributing an algorithm represented as a graph across processing units

- social networks

- ↳ Find tightly connected pieces in "friend" graph
 - ↳ for recommendations, directed advertising